

Consider this example in computed property:

```

• // ...
• computed: {
•   fullName: {
•     // getter
•     get: function () {
•       return this.firstName + ' ' + this.lastName
•     },
•     // setter
•     set: function (newValue) {
•       var names = newValue.split(' ')
•       this.firstName = names[0]
•       this.lastName = names[names.length - 1]
•     }
•   }
• }
• }
• // ...

```

Computed properties just behave like getters. Whenever you access it, the getter function re-evaluates. Computed properties are cached only when one of its reactive dependencies has been changed.

Let's take an example. Suppose you have an expensive computed property named **A**. It will require looping through an array, and a lot of computation will be involved. Now, we have another computed property dependent on **A**. If it is not cached, we will call it getter many more times than required. This would potentially cause issues with performance. If it is cached, its dependencies will not be changed, and you can access it many times that will further not trigger any unnecessary computations. Furthermore, it is very important to understand what 'reactive dependency' means.

```

• var vm = new Vue({
•   data: {
•     msg: 'hi'
•   },
•   computed: {
•     example: function () {
•       return Date.now() + this.msg
•     }
•   }
• })

```